

Predicting returns, and what coverage should cost

Morgan Cooper · how I approached pricing returns coverage, on 2.3M real order lines

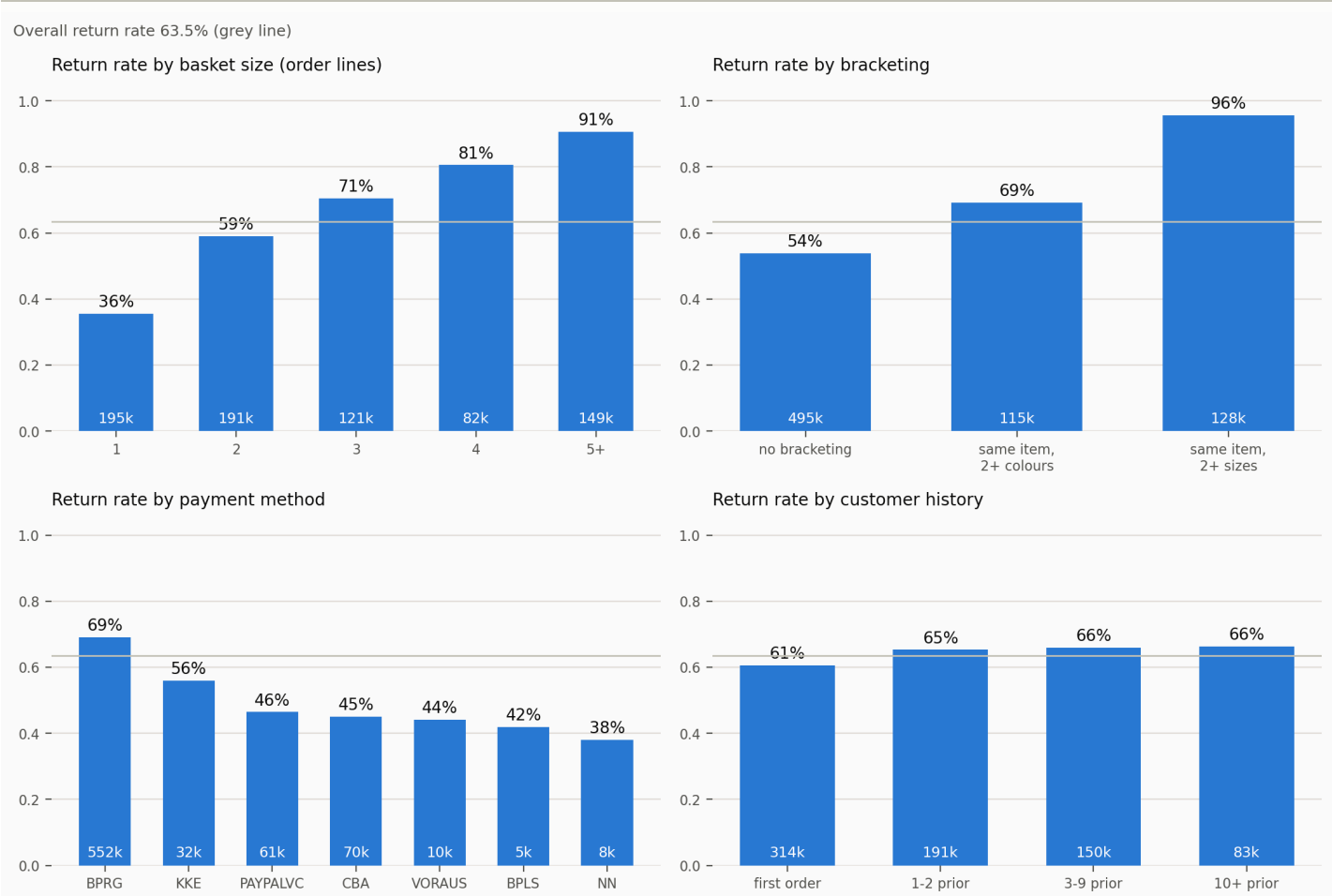
How I went about it

Paid returns coverage is an insurance product sold at checkout, and it lives or dies on one number per order: how likely this order is to come back. If that number is trustworthy in absolute terms — not just good at ranking — the price follows from it. If it is not, every price is a guess.

I used the Data Mining Cup 2016 set: **2,325,165 order lines** from an anonymised European fashion retailer. I chose it over the usual Kaggle options because it carries a **real return label**, so I would not spend this memo defending a proxy invented from review scores. Collapsing the lines gives **738,698 orders**, labelled by whether anything came back — the unit a policy actually pays out on. This book returns **63.5% of orders**, the global high-water mark for fashion, so the pricing is swept across gentler return rates at the end.

I trained on the 624,732 orders before 2015-07-01 and scored the 113,966 that followed, so the model is only ever predicting forward. Nothing a customer did after an order was placed is allowed to inform that order's features. I unit-tested the helper enforcing that against a hand-worked example, because my first version of it was silently wrong and still produced entirely plausible numbers.

What I found in the data



Four things visible the moment an order is placed. Grey line is the 63.5% overall rate; counts are printed so a tall bar built on few orders cannot pass as signal.

Bracketing is the clearest signal in the dataset. When a shopper orders the **same article in two or more sizes**, something comes back **96%** of the time — against 54% for orders with no bracketing at all. Nobody wears both. Basket size runs the same way: a single-line order returns 36% of the time, a five-line order 91%.

Payment method spreads thirty points top to bottom, almost certainly reflecting how committed the shopper is at checkout rather than anything about the payment rail itself. Customer history barely moves the needle on its own — and then turns out to be the strongest feature the model has. Worth knowing before anyone dismisses a feature for looking flat in isolation.



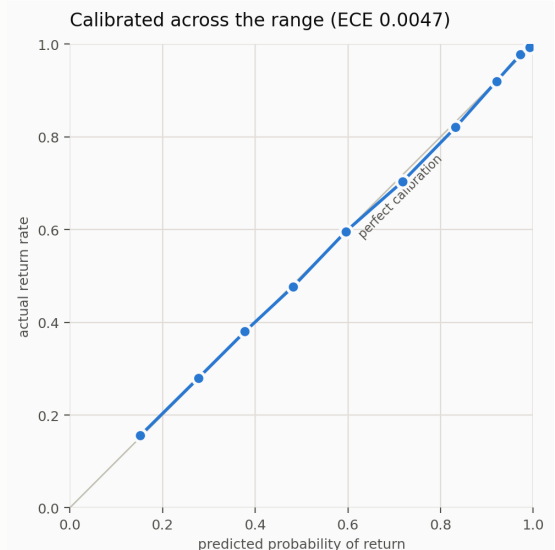
The left panel is the honest picture: no single view of the raw data pulls returns apart from keeps. The signal is real but only emerges once features are combined, which is what the model is for. That clustering finds a 48-point spread *without ever seeing the outcome* is good evidence the structure is in the data rather than invented by the model.

What the model says

Model	AUC	Log loss	Calibration error
Raw order attributes (baseline)	0.7959	0.5218	—
Logistic regression, all features	0.8348	0.4819	—
LightGBM, all features	0.8526	0.4527	0.0047
XGBoost, all features	0.8528	0.4524	0.0050

Calibration error is 0.0047. The riskiest tenth of orders is predicted at 0.993 and comes back 0.994 of the time. This is the result the pricing rests on: quote someone 0.40 and they really do return 40% of the time, so that number can be turned into dollars without distortion.

XGBoost beats LightGBM by **0.0002**, and the two agree on 99.8% of what they predict. More performance is not hiding in a different algorithm. Logistic regression gets within 0.018 of both, which says the engineered features are doing the work rather than the tree.



Predicted against actual, 10 bins, validation set.

The customer's own return history is worth **four times more than any other feature**; bracketing is second. The model's weak spot is equally clear: on **single-line orders it scores 0.696 against 0.858** on multi-line, because with no basket to read it has only the customer's history to go on. Two small payment segments also drift far enough off calibration that I would recalibrate them before trusting their prices.

How I would price from this

A return costs the platform **\$8** to ship back and process. Each shopper is then quoted their own expected cost, marked up to a target margin:

$$\text{price} = (\text{probability of return} \times \$8) \div (1 - \text{margin})$$

Because the probability is trustworthy in absolute terms, this earns the target margin on *every single order* — the safest shopper and the riskiest alike. Nothing is tuned, and no assumption about shopper behaviour enters the price.

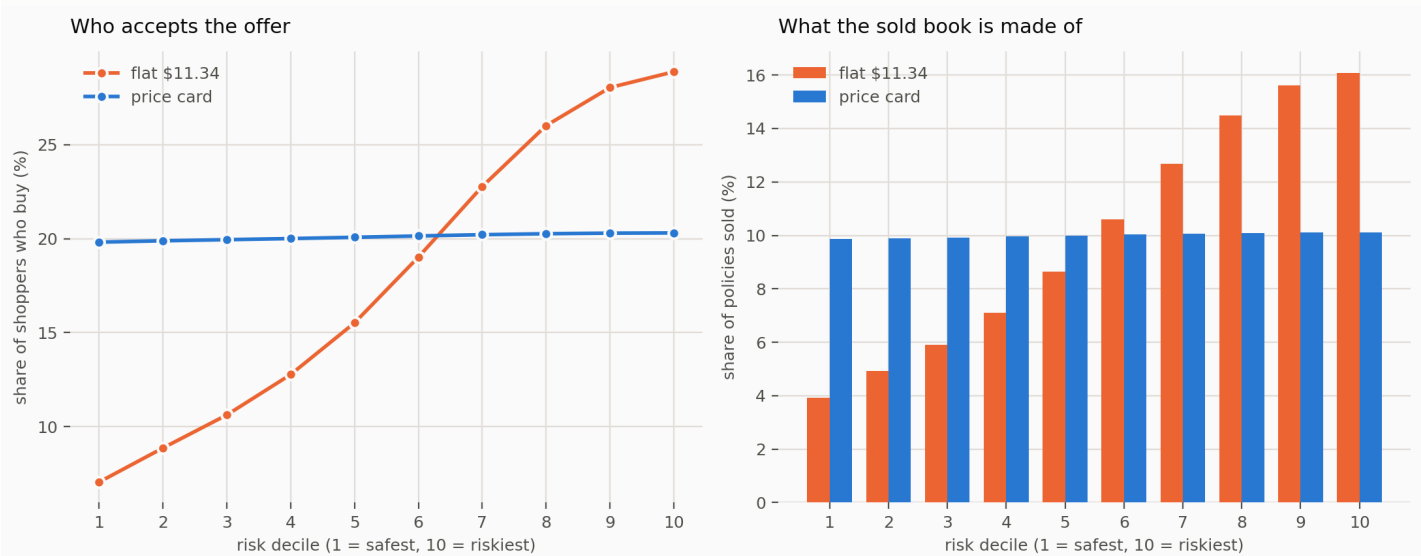
Risk decile	Return probability	Expected claim	Price at 45%	Price at 60%	Earns at 45%
1 (safest)	0.15	\$1.89	\$3.44	\$4.73	\$1.55
3	0.38	\$3.52	\$6.40	\$8.80	\$2.88
5	0.60	\$5.09	\$9.25	\$12.72	\$4.16
8	0.92	\$7.43	\$13.51	\$18.57	\$6.08
10 (riskiest)	0.99	\$7.95	\$14.45	\$19.87	\$6.50

One flat price earning that same 45% across the whole book would be **\$11.34**. Against it, the card sells to **20.1% of shoppers rather than 18.0%** at identical profitability — more customers, same margin, because the safe majority is no longer quoted a price built for somebody else's risk.

A high quote at the top carries no risk. A shopper who declines pays their own return shipping, so the platform has no revenue and no claims from them. Quoting the riskiest decile \$14.45 costs nothing if they walk. What it does cost is optics — \$19.87 on a \$106 order is nearly a fifth of the cart, and that is a merchant conversation.

Who actually buys, and why that is the real win

The obvious objection is that the riskiest shoppers will see \$14.45 and decline. They do — and that is the point.



Under one flat price, uptake climbs with risk and the sold book concentrates in the riskiest deciles. Under the card, uptake is nearly identical across all ten.

	Flat \$11.34	Price card
Buyers in the safest decile	7.1%	19.8%
Buyers in the riskiest decile	28.9%	20.3%
Average risk of the people who buy	0.755	0.634
Versus the population at 0.632	+0.123	+0.002
What a sold policy costs to service	\$6.24	\$5.37

One flat price is a bargain for the shopper who knows they will return and a bad deal for the one who knows they will not, so the book fills up with claimants: buyers run **+0.123 riskier than the population** and cost \$6.24 each to service. Quote everyone their own expected cost and that arbitrage disappears — uptake lands between **19.8% and 20.3% across all ten deciles**, and the gap between buyers and everyone else collapses to **+0.002**. Servicing a policy gets 14% cheaper.

The safest decile nearly triples its uptake and the riskiest falls by nearly a third, and that shift is carried through every revenue and cost figure in this memo — each is built from who actually buys at the price they are shown, not from an

assumption that everyone takes the offer equally. **Adverse selection is not being tolerated here, it is being priced away.**

Scaled to \$100M

None of this is specific to this retailer, so the same rule can be run against a book with any return rate. Two things fall out.

The economics do not move. Every order earns the target margin, so a **\$100M coverage book returns \$45M at a 45% margin — whatever the return rate.** One number covers every scenario. **The prices do move:** what changes is the range the card has to span to get there.

Return rate	Average claim	Flat price at 45%	Card range at 45%	Card range at 60%	Contribution on \$100M
10%	\$1.52	\$3.26	\$1.47 – \$8.92	\$2.03 – \$12.27	\$45M
15%	\$1.88	\$4.30	\$1.50 – \$10.98	\$2.06 – \$15.10	\$45M
20%	\$2.24	\$5.36	\$1.53 – \$12.28	\$2.11 – \$16.89	\$45M
25%	\$2.60	\$6.37	\$1.59 – \$13.09	\$2.18 – \$17.99	\$45M
30%	\$2.96	\$7.31	\$1.66 – \$13.58	\$2.29 – \$18.68	\$45M
40%	\$3.68	\$8.88	\$1.92 – \$14.10	\$2.64 – \$19.38	\$45M
50%	\$4.40	\$10.09	\$2.36 – \$14.32	\$3.25 – \$19.69	\$45M
63% (this book)	\$5.35	\$11.34	\$3.44 – \$14.45	\$4.73 – \$19.87	\$45M

At a 20% return rate the one-size price would be \$5.36, while the card spans \$1.53 to \$12.28 — the safest decile pays a dollar fifty, the riskiest just over twelve. Note the top of the range stays high even on a gentle book: a 10%-return merchant still has a tenth of its shoppers returning most of what they buy, and they are still priced for it. That is the case for pricing the order rather than the merchant.

The rule needs nothing tuned per book and nothing re-tuned as behaviour drifts — only a trustworthy probability and the cost of a return.

Measured from the data: every return outcome, all model results, and the claim costs derived from them. Assumed: that a return costs \$8 to process. The price card rests on nothing else. Figures for uptake and revenue additionally use a demand curve anchored at 35% of shoppers buying at \$5, set in one place and easy to change. Margin throughout is measured on revenue. Gentler return rates are modelled by shifting this book's probabilities down while preserving the model's ranking. Amounts in dollars; the source retailer is euro-denominated, treated 1:1. Code and notebooks: github.com/cooper-rm/ecommerce-returns-ml